

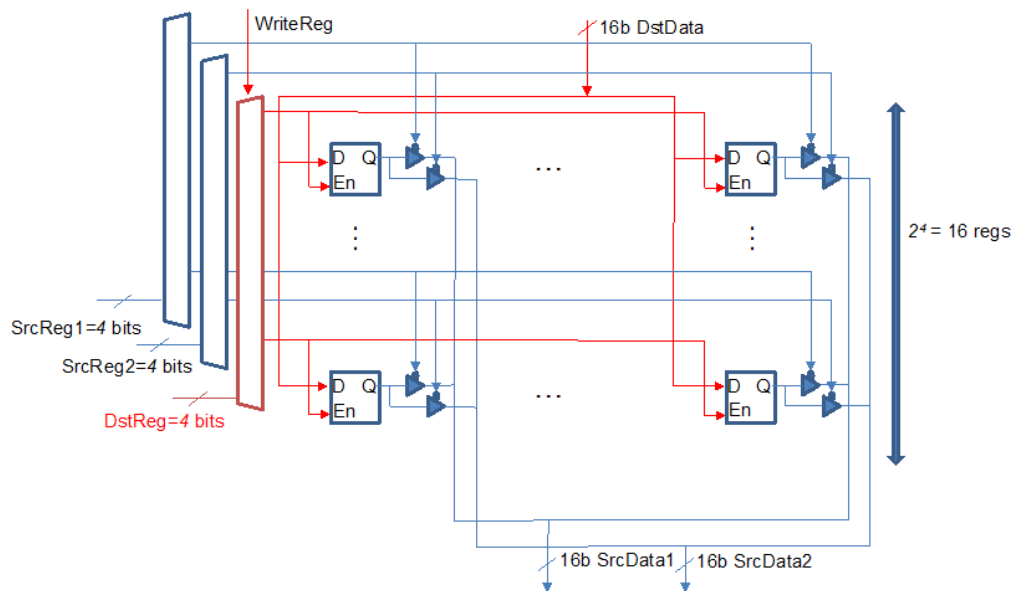
Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 4 (Due 10/08)

For the question below (Verilog based), paste your entire Verilog code in a single text/doc file and submit the text/doc file on Canvas. Note that these Verilog questions need to follow the Verilog rules specified in the 552 project/Verilog rules document that can be found on Canvas.

1. Register File

You are required to design a register file with 16 registers, each 16-bits wide. The register file should have two read ports and one write port. The expected design is shown in the figure below:

2R1W 16x16 Register File



- i. From the figure, there are two 4-bit source registers inputs and one 4-bit destination register input to perform 2 reads and/or 1 write.
- ii. The register file is made up of 16 registers and each register is made of 16 bit-cells.
- iii. Each bit cell (as show above) consists of a D-Flip Flop and tri-state buffers. The D Flip-Flop is provided to you (refer to dff.v included within *project-phase1-env.zip* in Canvas's Source Project module).
- iv. The respective register information is decoded via the 4-16 decoders, which you are expected to implement using assign statements and operators (as opposed to an exhaustive case statement).
- v. Once a read register is decoded, the particular register (i.e., a row of 16 bitcells) is enabled. The enabling is performed via the tri-state buffers (as shown in the figure).

- vi. Once a write register is decoded, the particular register (row of bitcells) is given the write-enable and dst_data is written into it.
- vii. ~~You are expected to design the register file to support internal write before read bypassing. This means that your design should be able to read the written value to a register in the same cycle as a write. Logic to perform bypassing is not shown in the figure.~~ **NOT**

UNTIL PHASE2

The required modules are listed below:

```
module read_decoder_4_16(  
    input [3:0] reg_id,  
    output [15:0] wordline);
```

```
module write_decoder_4_16(  
    input [3:0] reg_id,  
    input write_reg,  
    output [15:0] wordline);
```

```
module bit_cell(  
    input clk,  
    input rst,  
    input d,  
    input wren,  
    input rden1,  
    input rden2,  
    inout bitline1,  
    inout bitline2);
```

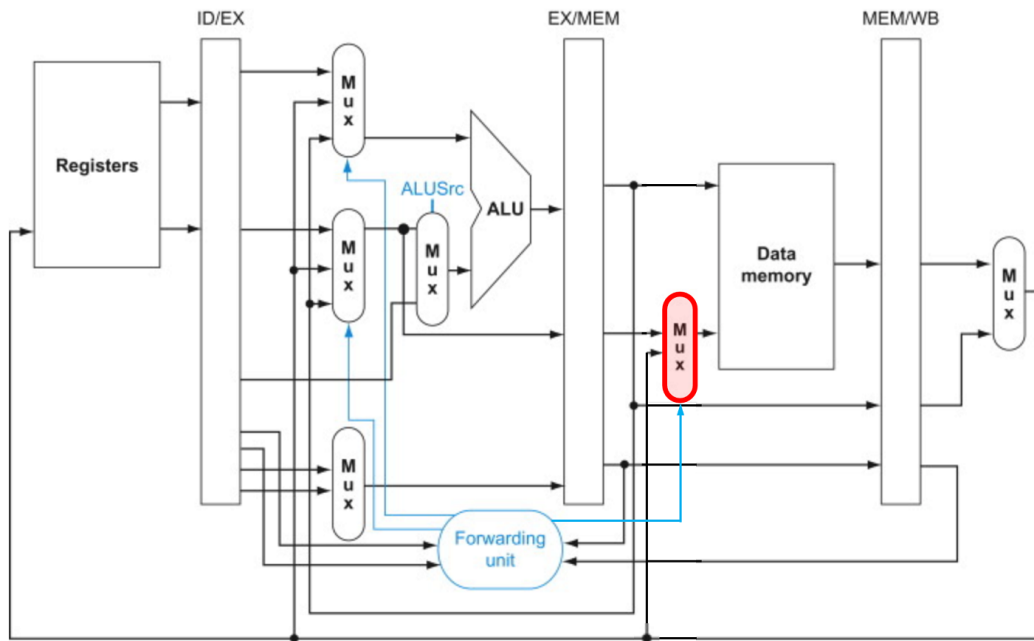
```
module register(  
    input clk,  
    input rst,  
    input [15:0] d,  
    input write_reg,  
    input rden1,  
    input rden2,  
    inout [15:0] bitline1,  
    inout [15:0] bitline2);
```

```
module register_file(  
    input clk,  
    input rst,  
    input [3:0] src_reg1,  
    input [3:0] src_reg2,  
    input [3:0] dst_reg,  
    input write_reg,  
    input [15:0] dst_data,  
    inout [15:0] src_data1,  
    inout [15:0] src_data2);
```

The following question is non-verilog. Submit this as a text doc or a snapshot.

2. MEM-to-MEM Forwarding

Consider MIPS store instructions of the form: `sw $rt, imm($rs)`. As in other instructions, register `$rs` is needed in the EX stage. But register `$rt` is not needed until the MEM stage. Thus load-to-use hazard stalls for `$rt` can be eliminated by adding support for MEM-to-MEM forwarding. This involves adding a forwarding mux at the write port of data memory, as shown below (modified from COD Figure 4.57).



Assume the five-stage pipeline with full forwarding and bypassing but now also with support for MEM-to-MEM forwarding. Note that as in the textbook, RegisterRd below refers to the destination register for either an R-type instruction (which comes from the instruction's Rd) or an I-type instruction (which comes from instruction's Rt).

- (a) Fill in the pipeline timing diagram for the sequence of instructions below. Use "*" to indicate a pipeline stall at the clock cycle of the corresponding instruction.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
<code>lw \$s1, 0(\$s2)</code>	F	D	X	M	W										
<code>sw \$s1, 4(\$s3)</code>		F	D	X	M	W									
<code>lw \$s1, 4(\$s2)</code>			F	D	X	M	W								
<code>sw \$s4, 8(\$s1)</code>				F	D*	D	X	M	W						

- (b) Provide the necessary conditions for setting the MEM-to-MEM forwarding mux.

Answer:

**if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
 and (MEM/WB.RegisterRd = EX/MEM.RegisterRt)
) enable MEM-to-MEM forwarding;**

- (c) Modify the conditions below for load-to-use stalls now that MEM-to-MEM forwarding is supported.

**if (ID/EX.MemRead and (ID/EX.RegisterRd $\neq$ 0)
 and ((ID/EX.RegisterRd = IF/ID.RegisterRs) or (ID/EX.RegisterRd = IF/ID.RegisterRt))
) enable load-to-use stall;**

Answer:

**if (ID/EX.MemRead and (ID/EX.RegisterRd $\neq$ 0)
 and ((ID/EX.RegisterRd = IF/ID.RegisterRs)
 or ((ID/EX.RegisterRd = IF/ID.RegisterRt) and not IF/ID.MemWrite))
) enable load-to-use stall;**